

Universitatea Națională de Știință și Tehnologie Politehnica București

Automatică și Calculatoare / Ingineria Sistemelor

CueUp

Documentație tehnică

Realizat de: Gorgovan Tudor
Mustață Bianca-Andreea

București, 2026

Cuprins

1	Introducere	1
2	Obiective și motivație	1
2.1	Motivația proiectului	1
2.2	Obiective	1
3	Descrierea publicului țintă	1
4	Cerințe funcționale și nefuncționale	2
4.1	Cerințe funcționale	2
4.2	Cerințe nefuncționale	2
5	Tehnologii utilizate și logica aplicației	2
5.1	Tehnologii utilizate	2
5.2	Logica din spatele aplicației	3
6	Procesul de lucru pas cu pas	3
6.1	Generarea setului de date	3
6.2	Modelul de date și încărcarea prin ETL	4
6.3	Implementarea interfeței	5
6.4	Implementarea backend-ului	8
6.5	Testarea și validarea	11
7	Descrierea rapoartelor	12
8	Dificultăți întâmpinate	13
9	Concluzii	14
	Bibliografie	14

1 Introducere

CueUp este un proiect realizat în cadrul școlii de vară „Data Journey From Cloud to AI”. Acesta simulează o interfață ce are ca rol ajutarea utilizatorilor de a își găsi oponenți cu care să se joace biliard.

Documentația prezintă obiectivele proiectului, publicul țintă, cerințele, tehnologiile folosite, etapele de implementare și rezultatele obținute.

2 Obiective și motivație

2.1 Motivația proiectului

Ideea proiectului a apărut din motivația de a ajuta persoanele doritoare să se joace biliard chiar și atunci când nu au cu cine.

Soluțiile existente necesită depunerea unui efort ridicat și prezintă riscuri, existând posibilitatea de a nu se găsi fie oponent doritor, fie o masă disponibilă la momentul dorit.

Problema are și o a doua latură, cea a cluburilor. Venitul acestora vine din orele de masă vândute, iar costurile sunt fixe. Interesul cluburilor este de a avea mesele rezervate cât mai mult.

2.2 Obiective

Principalele obiective ale proiectului sunt:

- dezvoltarea unei interfețe intuitive și ușor de utilizat de oricine;
- automatizarea procesului de găsim a oponenților din același oraș și la un nivel apropiat față de cel al utilizatorului;
- opțiunea rezervării meselor de biliard;
- oferirea sfaturilor de către o IA pentru îmbunătățirea performanței utilizatorilor;
- afișarea clasamentelor locale și naționale;
- punerea la dispoziția cluburilor a unor rapoarte despre ocupare și venituri.

3 Descrierea publicului țintă

Aplicația se adresează în principal următoarelor categorii de utilizatori:

- **Jucători competitivi**, care joacă săptămânal și au nevoie de un adversar de nivel apropiat, nu doar de o masă liberă;
- **Jucători ocazionali și grupuri sociale**, care ies de câteva ori pe lună, de obicei vineri sau sâmbătă seara, și vor certitudinea că masa este disponibilă;
- **Cluburi de biliard**, din interesul pentru rezervarea cât mai frecventă a meselor disponibile, pentru popularizarea locului și pentru datele despre propria activitate.

4 Cerințe funcționale și nefuncționale

4.1 Cerințe funcționale

- CF1. Sistemul trebuie să permită autentificarea utilizatorilor.
- CF2. Utilizatorul trebuie să poată adăuga, modifica și șterge date.
- CF3. Sistemul trebuie să permită căutarea și filtrarea informațiilor.
- CF4. Sistemul trebuie să permită rezervarea meselor și căutarea oponentilor.
- CF5. Sistemul trebuie să ofere recomandări personalizate, pe baza istoricului de joc.
- CF6. Aplicația trebuie să pună la dispoziție rapoarte despre activitatea cluburilor și a comunității.

4.2 Cerințe nefuncționale

- CNF1. Interfața trebuie să fie intuitivă și adaptabilă la diferite ecrane.
- CNF2. Datele utilizatorilor trebuie protejate împotriva accesului neautorizat.
- CNF3. Timpul de răspuns pentru operațiile uzuale trebuie să fie redus.
- CNF4. Aplicația trebuie să fie ușor de întreținut și extins.
- CNF5. Sistemul trebuie să ofere mesaje clare în cazul apariției erorilor.
- CNF6. Integritatea datelor trebuie garantată în baza de date, indiferent de sursa care le introduce.

5 Tehnologii utilizate și logica aplicației

5.1 Tehnologii utilizate

Generarea și pregătirea datelor

- Claude Opus 5 High pentru generarea setului de date sintetic
- Python, pentru scriptul de generare
- Fișiere CSV ca format intermediar de schimb

Stocare și ETL

- Microsoft Azure SQL pentru baza de date CueUp
- SQL Server Integration Services (SSIS), dezvoltat în Visual Studio, pentru pipeline-ul de încărcare

Aplicație

- Python 3.13
- Streamlit, framework pentru interfața web
- pyodbc pentru conectarea cu SQL Server, prin ODBC Driver 17
- `python-dotenv` pentru gestionarea credențialelor din fișiere `.env`
- `google-genai` pentru API-ul Gemini

Raportare

- Microsoft Power BI Desktop

Control versiune

- Git și GitHub

5.2 Logica din spatele aplicației

Există mai multe etape ce asigură funcționarea corectă a platformei:

1. **Generarea datelor sintetice.** Un script Python scris cu Claude Opus 5 produce șapte fișiere CSV, cu tipare realiste de cerere.
2. **ETL în SSIS.** Un pachet cu un task de creare a schemei și șapte fluxuri de date înlănțuite.
3. **Popularea tabelelor.** Datele ajung în Azure SQL, în șapte tabele legate prin nouă chei străine și protejate de constrângeri CHECK.
4. **Interfața Streamlit.** Autentificare, căutare de mese libere, rezervare, parteneri, clasament și antrenor AI. Interfața primește datele de la utilizator, le prelucrează și afișează rezultatul.
5. **Rapoarte Power BI.** Două rapoarte construite peste același model relațional.

Aplicația și rapoartele citesc din aceeași bază, deci nu există copii ale datelor. Din interfață pleacă și apelul către API-ul Gemini, alimentat cu statisticile reale ale jucătorului.

Principalele module de Python folosite sunt `streamlit`, `pandas`, `os`, `sys`, `dotenv`, `pyodbc` și `google-genai`. Acestea asigură de la prelucrarea, respectiv actualizarea bazei de date, până la interacțiunea utilizatorului cu instanța IA.

6 Procesul de lucru pas cu pas

6.1 Generarea setului de date

Setul de date a fost generat de Claude Opus 5, printr-un script Python care produce șapte fișiere CSV, câte unul pentru fiecare tabelă.

Perioada conținutului generat este acoperită de la 26 ianuarie 2026 până la 16 august 2026, iar volumele sunt cele din Tabelul 1.

Tabelă	Rânduri
Orașe	10
Cluburi	18
Mese	111
Jucatori	2.500
Rezervari	44.259
Meciuri	17.514
IstoricRating	21.572

Tabela 1: Volumele setului de date generat

Rating-urile ELO nu au fost generate direct. Fiecare jucător pornește de la o valoare extrasă dintr-o distribuție normală, iar meciurile sunt procesate cronologic cu formula standard Elo, cu factorul K egal cu 24. Rezultatul fiecărui meci depinde de diferența de rating

Setul a fost verificat ulterior de diverse teste pentru asigurarea funcționării complete și corecte a aplicației și a bazei de date.

6.2 Modelul de date și încărcarea prin ETL

Modelul cuprinde șapte tabele. **Orașe**, **Cluburi** și **Mese** descriu oferta, **Jucatori** descrie cererea, **Rezervari** leagă cele două laturi, iar **Meciuri** și **IstoricRating** acoperă partea competitivă.

Integritatea este asigurată de nouă chei străine și de constrângeri **CHECK** care limitează valorile pentru **status**, **stare** și **tip_masa**, verifică ordinea datelor calendaristice și validează faptul că un câștigător este unul dintre cei doi jucători. Tabelele **Jucatori**, **Rezervari**, **Meciuri** și **IstoricRating** folosesc coloane **IDENTITY**, pentru ca aplicația să poată insera rânduri noi fără să genereze manual identificatori.

Încărcarea este un pachet SSIS unic. Un task **Execute SQL** creează schema de la zero, cu tot cu constrângeri și indecși. Scriptul este idempotent, deci baza poate fi reîncărcată oricând, fără curățare manuală.

Urmează un container secvențial cu șapte fluxuri de date, înlănțuite în ordinea impusă de cheile străine: Orașe, Cluburi, Mese, Jucători, Rezervări, Meciuiri, Istoric rating. Fiecare flux citește un fișier CSV și scrie într-o tabelă prin **OLE DB Destination**.

Un ultim task **Execute SQL** resincronizează contoarele coloanelor **IDENTITY**. Încărcarea în masă păstrează identificatorii din fișiere, dar nu actualizează contorul intern al bazei. Fără acest pas, prima înregistrare făcută din aplicație ar folosi un identificator deja existent.

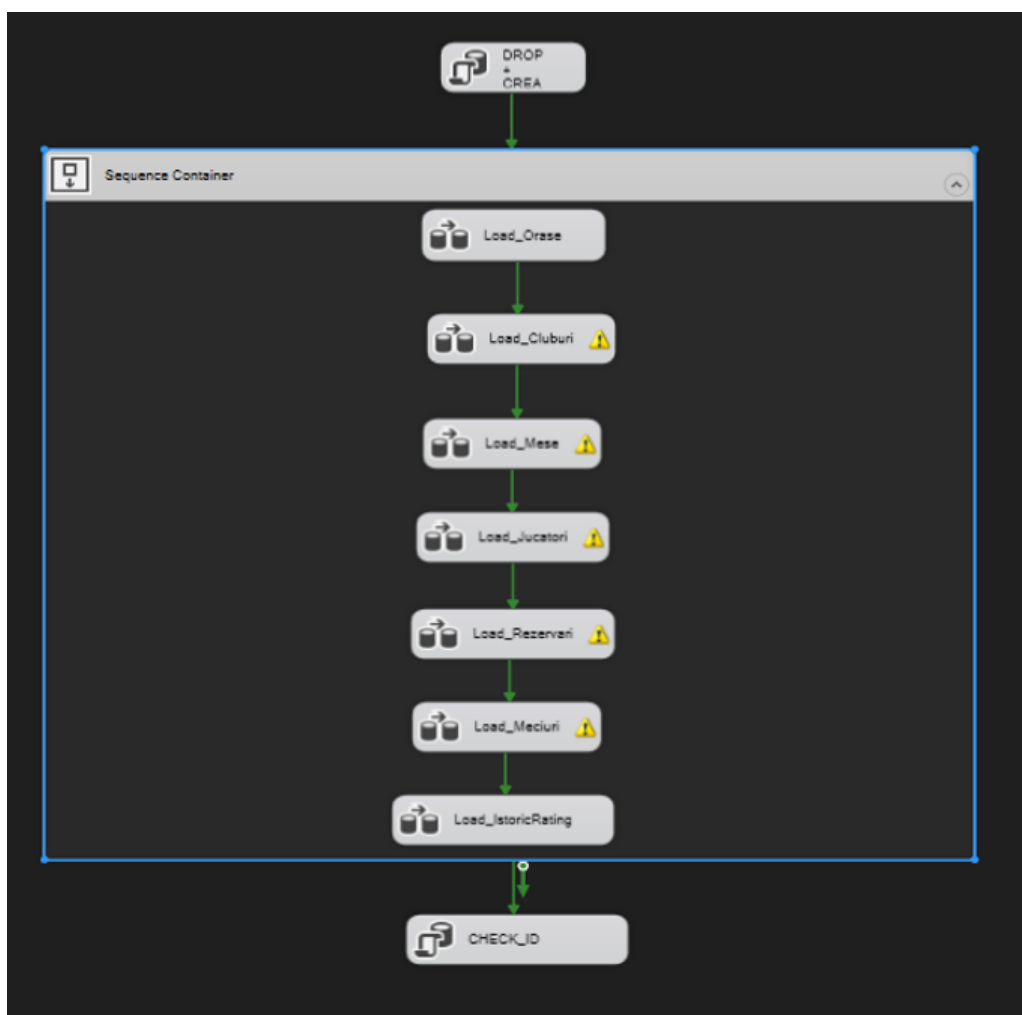


Figura 1: Control Flow-ul pachetului SSIS.

6.3 Implementarea interfeței

Odată identificată problema pe baza căreia a fost consolidat proiectul, a urmat etapa întrebărilor, pentru a forma o viziune generală a interfeței dorite. De la aspect la caracteristici, totul a fost analizat pentru a crea un produs cât mai intuitiv și folositor.

Pagina cu care este întâmpinat utilizatorul este pagina de autentificare.

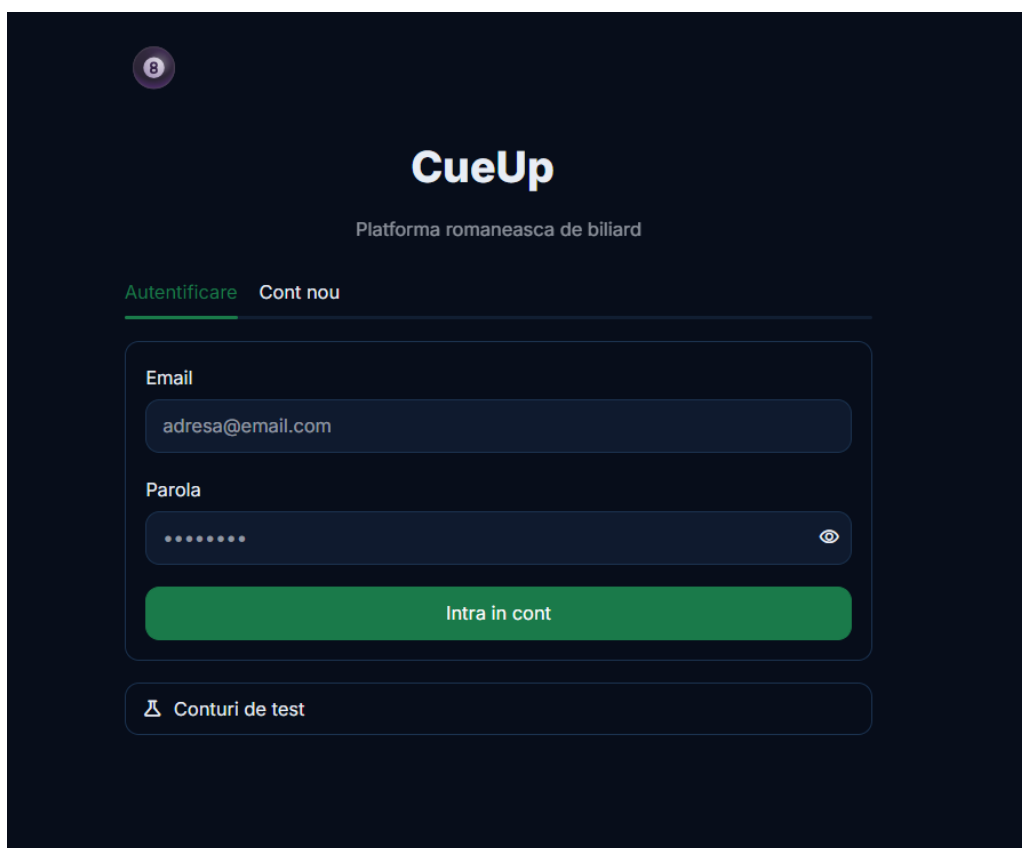


Figura 2: Pagina de autentificare a aplicației

Odată ce a fost realizată autentificarea, utilizatorul este redirecționat către pagina „acasă”, ce oferă informații generale legate de platformă.



Figura 3: Pagina de acasă

Din acest punct, utilizatorul este liber să navigheze oricare dintre pagini.

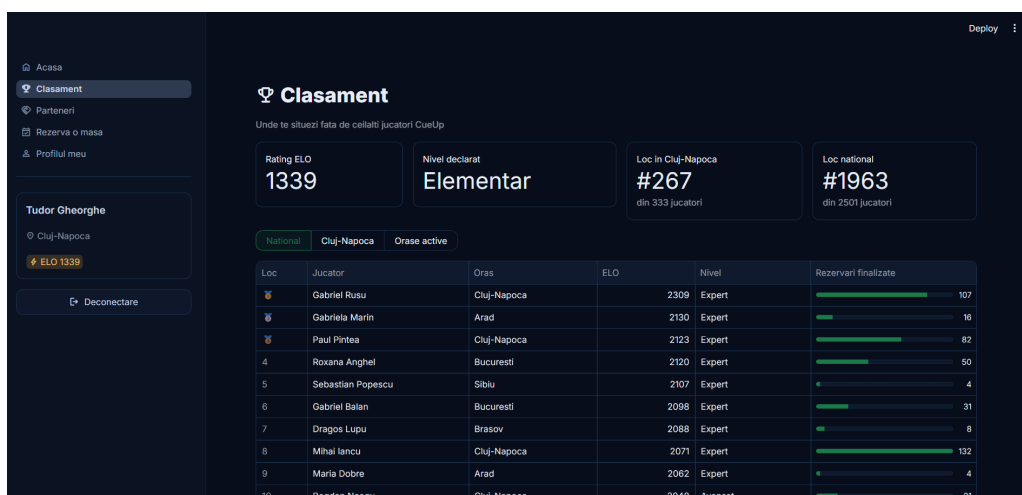


Figura 4: Pagina de clasament

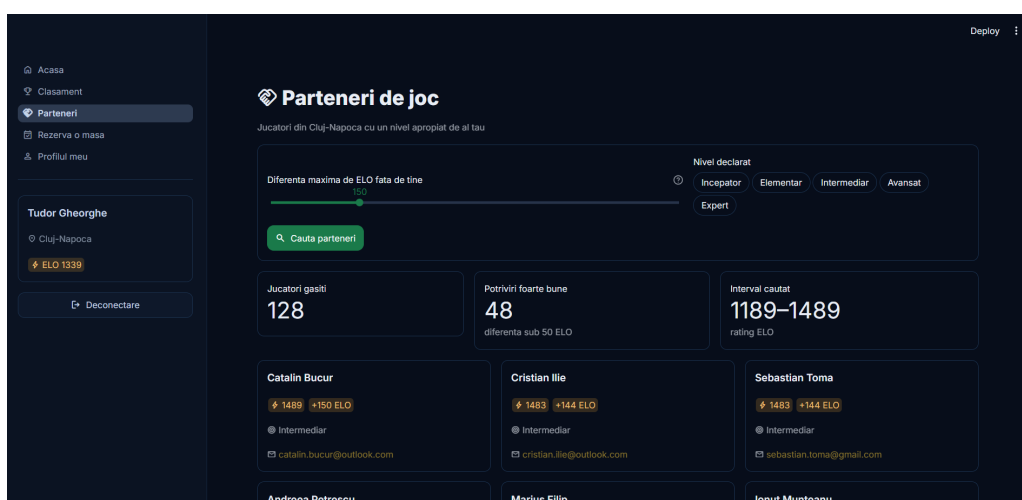


Figura 5: Pagina de parteneri

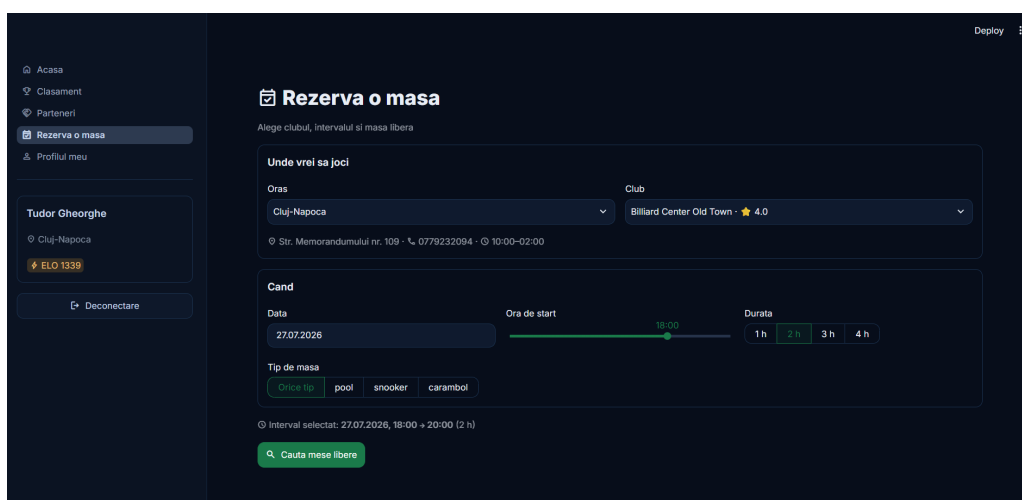


Figura 6: Pagina de rezervare a meselor

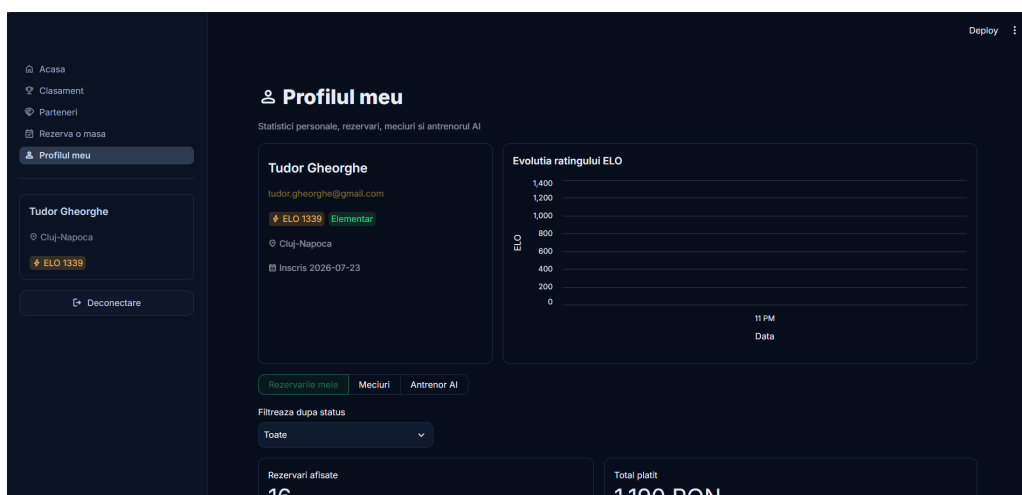


Figura 7: Pagina de profil

6.4 Implementarea backend-ului

Modulul de conexiune și interogări (`db.py`) concentrează toată comunicarea cu baza de date. Șirul de conexiune se construiește din variabile de mediu citite dintr-un fișier `.env`, care nu este versionat. Fiecare interogare își deschide și își închide propria conexiune, prin `contextlib.closing`. Managerul de context al bibliotecii `pyodbc` face doar commit sau rollback, nu închide conexiunea, iar fără acest pas Azure SQL ar refuza interogările următoare.

Rezultatele citirilor sunt ținute în cache prin `st.cache_data`, cu două durate: treizeci de minute pentru datele de referință și cinci minute pentru clasamente. Scrierile golesc cache-ul.

```

85  # — Q1: WHERE - Parteneri din acelasi oras cu ELO in interval —
86  @st.cache_data(ttl=TTL_STATISTICI, show_spinner=False)
87  def cauta_parteneri(id_oras: int, elo_min: int, elo_max: int,
88                      exclude_id: int) -> pd.DataFrame:
89      sql = f"""
90          SELECT j.id_jucator, j.nume, j.email,
91                 j.nivel_declarat, j.rating_elo,
92                 j.data_inscriere, o.nume_oras
93          FROM   Jucatori j
94          JOIN   Orase o ON j.id_oras = o.id_oras
95          WHERE  j.id_oras = {_nr(id_oras)}
96                 AND j.rating_elo BETWEEN {_nr(elo_min)} AND {_nr(elo_max)}
97                 AND j.id_jucator != {_nr(exclude_id)}
98          ORDER BY j.rating_elo DESC
99      """
100     return _df(sql)

```

Figura 8: Fragment din fișierul `db.py`

Construirea interogărilor. Instrucțiunile SQL sunt scrise ca șiruri formatate, iar valorile venite din interfață trec prin două funcții ajutătoare. `_nr` convertește valoarea la

întreg sau zecimal, deci respinge orice nu este numeric. `_txt` pune valoarea între apostrofuri și le dublează pe cele din interior, așa cum cere SQL Server pentru literalii de tip text. Cele două funcții sunt singurul loc unde se tratează valorile, deci regula se aplică uniform în toate interogările.

Cele cinci interogări obligatorii sunt definite tot în `db.py` și apar în Tabelul 2. Trei filtrează la nivel de rând, prin `WHERE`, iar două la nivel de grup, prin `HAVING`.

Tip	Funcție	Rol
WHERE	<code>cauta_parteneri</code>	Jucători din același oraș, cu rating într-un interval ales, fără cel curent
WHERE	<code>mese_active_club</code>	Mesele active dintr-un club, cu filtru opțional pe tip
WHERE	<code>rezervari_jucator</code>	Rezervările unui jucător, filtrate opțional după status
HAVING	<code>orase_active</code>	Orașele cu cel puțin un număr dat de jucători cu rezervări finalizate
HAVING	<code>clasament_jucatori_activi</code>	Jucătorii cu peste un număr dat de rezervări finalizate

Tabela 2: Cele cinci interogări obligatorii, definite în `db.py`

Ultimele două folosesc `HAVING` pentru că filtrează valori care apar abia după agregare. Numărul de jucători activi dintr-un oraș nu poate fi știut înainte de grupare, deci condiția nu poate sta într-un `WHERE`.

Modulul de autentificare (`auth.py`) expune trei funcții, care apelează `db.py`: verificarea perechii email și parolă, verificarea unui email deja înregistrat și crearea unui cont nou. La eșec, mesajul afișat este identic indiferent dacă emailul sau parola este greșită, ca să nu se poată afla ce conturi există. Starea de autentificare se păstrează în `st.session_state`.

```
6 import sys, os
7 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
8 from db import verify_login, email_exists, register_player
9
10
11 def verify_user(email: str, parola: str):
12     """
13     Verifică credențialele direct în Azure SQL.
14     Returnează dict-ul jucătorului dacă sunt corecte, altfel None.
15     """
16     return verify_login(email.strip(), parola)
17
18
19 def email_registered(email: str) -> bool:
20     """Returnează True dacă email-ul există deja în Jucatori."""
21     return email_exists(email.strip())
22
23
24 def register_user(
25     nume: str,
26     email: str,
27     parola: str,
28     id_oras: int,
29     nivel: int,
30 ) -> bool:
31     """
32     Înregistrează un jucător nou în Azure SQL.
33     Returnează True la succes, False dacă email-ul există deja sau INSERT eșuează.
34     """
35     email = email.strip()
36     if email_exists(email):
37         return False
```

Figura 9: Fragment din fișierul auth.py

Modulul de integrare AI (`gemini_client.py`) construiește promptul pentru API-ul Gemini și tratează erorile de rețea printr-un răspuns de rezervă. În prompt intră rating-ul actual și evoluția lui, numărul de meciuri finalizate, rata de victorie și nivelul declarat, astfel încât sfatul primit să se refere la situația reală a utilizatorului. Modelul nu generează cod SQL: primește date și întoarce text.

```
91 def _construieste_prompt(  
92     nume: str,  
93     rating_elo: int,  
94     nivel: int,  
95     win_rate: float,  
96     nr_meciuri: int,  
97     intrebare: str = "",  
98     evolutie_elo: int | None = None,  
99     ultimele_meciuri: list[str] | None = None,  
100 ) -> str:  
101     """Construieste prompt-ul cu statisticile reale ale jucatorului."""  
102     nivel_text = _NIVEL_MAP.get(nivel, "intermediar")  
103  
104     linii = [  
105         "Profilul real al jucatorului, din baza de date CueUp:",  
106         f"- nume: {nume}",  
107         f"- rating ELO actual: {rating_elo}",  
108         f"- nivel declarat: {nivel_text} ({nivel} din 5)",  
109         f"- meciuri finalizate: {nr_meciuri}",  
110         f"- rata de victorii: {win_rate:.0%}",  
111     ]  
112  
113     if evolutie_elo is not None:  
114         sens = "crescut" if evolutie_elo >= 0 else "scazut"  
115         linii.append(  
116             f"- evolutie ELO fata de primul meci inregistrat: a {sens} cu "  
117             f"{abs(evolutie_elo)} puncte"  
118         )  
119
```

Figura 10: Fragment din fișierul `gemini_client.py`

Validarea datelor se face pe două niveluri. În aplicație, formularele verifică formatul adresei de email, lungimea câmpurilor și intervalele valorilor numerice. În baza de date, constrângerile CHECK și cheile străine garantează integritatea indiferent de sursa datelor, inclusiv pentru încărcările prin ETL.

6.5 Testarea și validarea

Aplicația a fost testată prin scenarii care acoperă funcționalitățile principale. Fiecare buton, căsuță de text și slider a fost verificat astfel încât să producă rezultatul intenționat.

A fost testată și baza de date pentru verificarea adăugării datelor la momentul creării conturilor noi. Cazurile-limită și tratarea datelor incorecte au fost de asemenea gestionate, de exemplu autentificarea cu credențiale incorecte sau înregistrarea cu un email deja folosit.

Setul de date a fost validat separat, cu 39 de teste rulate înainte de încărcare, iar rezultatul încărcării a fost verificat prin compararea numărului de rânduri din fiecare tabelă cu valorile așteptate.

7 Descrierea rapoartelor

Primul raport, „Venituri și Ocupare”, conține patru indicatori sintetici (venit total, număr de rezervări, valoare medie a rezervării, jucători activi), o matrice de tip heatmap cu orele vândute pe zi a săptămânii și oră, evoluția lunară a veniturilor, distribuția veniturilor pe tip de masă și un clasament al cluburilor. Filtrele disponibile sunt orașul, clubul și tipul de masă, iar întreaga pagină este filtrată pe rezervările cu status **finalizata**, astfel încât anulările și neprezentările să nu distorsioneze cifrele de venit.

Heatmap-ul evidențiază un raport de aproximativ 53 la 1 între cel mai încărcat interval, sâmbătă la ora 19:00 cu 1.648 de ore vândute, și cel mai gol, miercuri la ora 9:00 cu 31 de ore. Gradul de ocupare este de 41,2% vineri și sâmbătă între orele 19 și 24, față de 6,9% de luni până joi între orele 10 și 17. Aceasta este observația centrală a proiectului: problema unui club de biliard nu este prețul, ci distribuția cererii în timp.

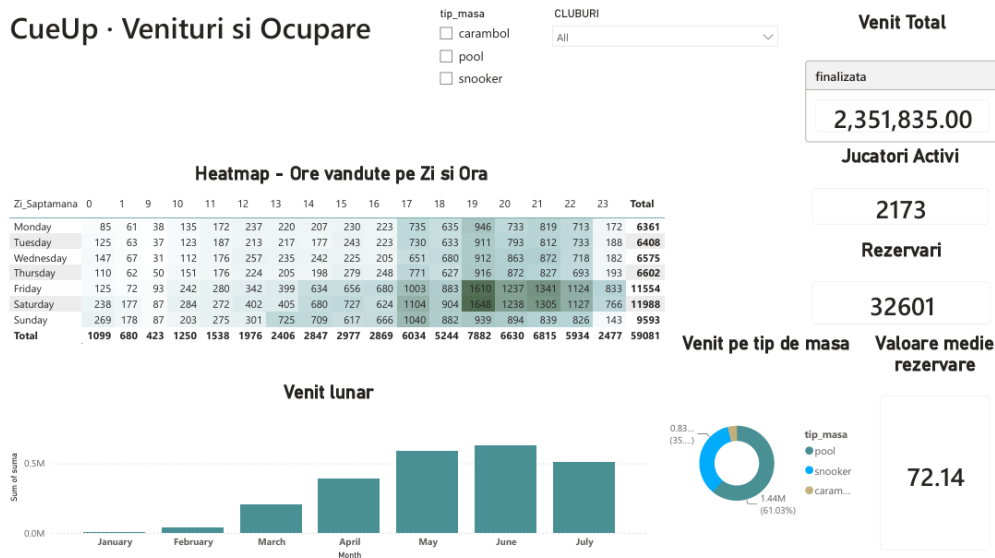


Figura 11: Raport 1

Al doilea raport, „Comunitate și meciuri”, conține distribuția rating-urilor ELO, relația dintre nivelul autodeclarat și rating-ul calculat, starea meciurilor, evoluția lunară a meciurilor private față de cele deschise și clasamentul jucătorilor.

Rating-ul mediu crește monoton cu nivelul autodeclarat, de la 1.199 pentru nivelul 1 până la 1.841 pentru nivelul 5, ceea ce confirmă că utilizatorii se autoevaluează corect în medie. Suprapunerea dintre categorii rămâne însă semnificativă, ceea ce justifică necesitatea unui rating calculat din rezultate, nu doar a unei autodeclarării.

CueUp · Comunitate si meciuri

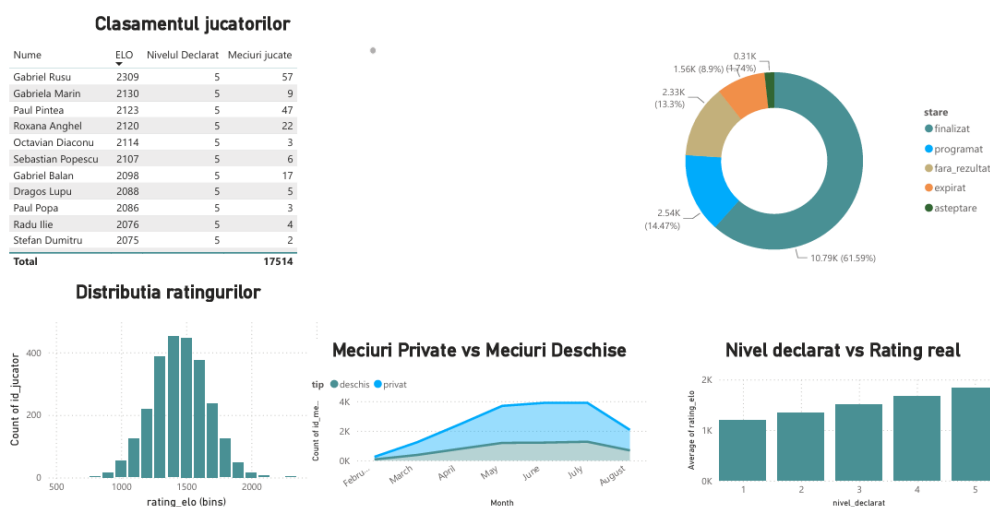


Figura 12: Raport 2

Ambele rapoarte se conectează la baza de date CueUp în mod Import și permit exportul vizualurilor în CSV, respectiv al paginii în PDF sau PowerPoint.

8 Dificultăți întâmpinate

Pe parcursul dezvoltării au fost întâmpinate patru dificultăți.

Protejarea datelor confidențiale. Atenția necesară pentru a nu împărtăși chei API sau parole ale bazei de date într-un depozit public.

Lecția învățată: folosirea fișierelor de tip `.env`, excluse prin `.gitignore`, și a unui fișier `.env.example` care arată ce chei sunt necesare, fără valori reale.

Pierderea valorilor NULL la încărcare. Tabela `Meciuri` are coloane care rămân goale în mod normal, de exemplu adversarul unui meci deschis la care nu s-a alăturat nimeni. Implicit, componenta `Flat File Source` din SSIS transformă câmpurile goale în zerouri, ceea ce ar fi atribuit 1.864 de meciuri unui jucător inexistent.

Lecția învățată: eroarea nu a dat niciun mesaj la încărcare. A fost prinsă de o constrângere `CHECK` din baza de date. Fără chei străine și constrângeri, datele s-ar fi încărcat aparent corect, iar problema ar fi apărut abia în rapoarte. Validarea trebuie să existe în baza de date, nu doar în aplicație.

Relații ambigue în modelul Power BI. La importul schemei din SQL Server, Power BI a creat automat relații pe toate cheile străine, inclusiv două căi între rezervări și orașe, una prin cluburi și alta prin jucători. Un model cu drumuri multiple între aceleași tabele dă rezultate imprevizibile, fără să semnaleze vreo eroare.

Lecția învățată: relațiile detectate automat trebuie verificate manual.

9 Concluzii

Proiectul a parcurs întregul lanț de prelucrare a datelor: un set sintetic generat cu ajutorul unui model IA, încărcat în Azure SQL printr-un pachet SSIS și folosit în paralel de aplicație și de cele două rapoarte. Setul cuprinde 44.259 de rezervări, 17.514 meciuri și 2.500 de jucători, din 18 cluburi și 10 orașe.

Dezvoltările ulterioare pot include plăți online, notificări trimise cu circa 24 de ore înainte de joc și extinderea către alte sporturi cum ar fi tenisul de masă.

Bibliografie

- [1] Streamlit Inc., *Streamlit Documentation*, <https://docs.streamlit.io/>
- [2] Google, *Gemini API Documentation*, <https://ai.google.dev/docs>
- [3] Python Software Foundation, *Python 3.13 Documentation*, <https://docs.python.org/3.13/>
- [4] Microsoft, *SQL Server Integration Services Documentation*, <https://learn.microsoft.com/en-us/sql/integration-services/>
- [5] Microsoft, *Power BI Documentation*, <https://learn.microsoft.com/en-us/power-bi/>
- [6] A. E. Elo, *The Rating of Chessplayers, Past and Present*, Arco Publishing, New York, 1978.